



# The Big Picture

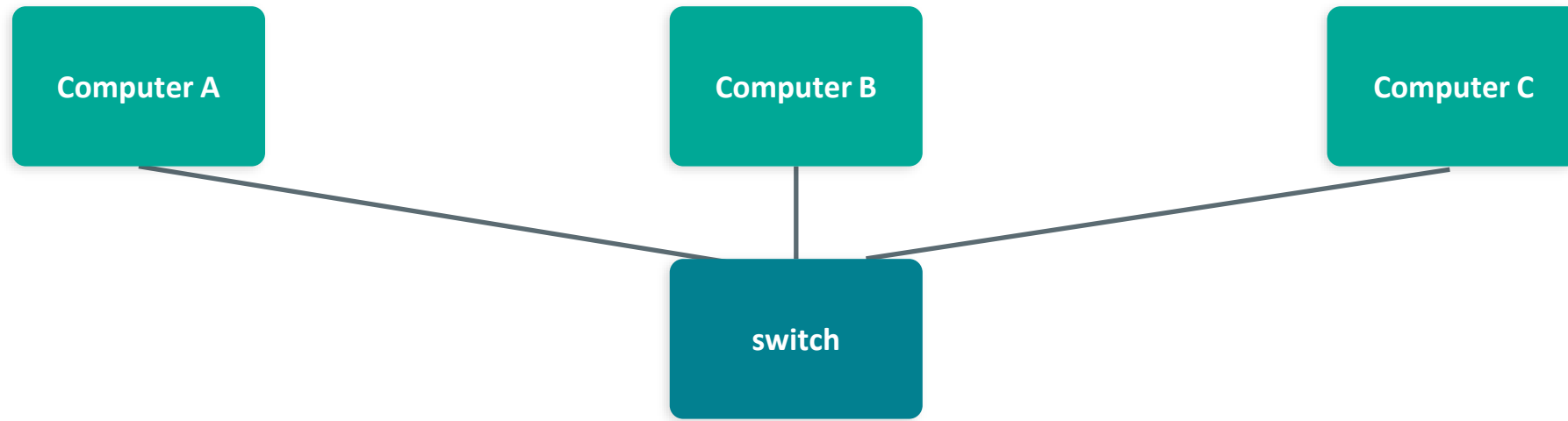
Before we touch Containerlab — a little intuition

*What is a virtual network? What is Docker? What is Containerlab — and why is it useful?*

No commands yet — just the ideas

# Start simple: what is a network?

A network is just **computers connected by links**, passing messages to each other.



*Each line is a cable. Each box can send and receive messages.*

To teach networking, students need their own network to experiment with — **but real cables, switches and routers are expensive and hard to share.**

# What is a virtual network?

The same idea — computers and links — but created entirely **in software**, running inside one machine.

## Physical network

- Real boxes, real cables
- Costs money per device
- Hard to reset or reshape
- One topology at a time

## Virtual network

- ✓ Software boxes, software links
- ✓ Free — runs on one computer
- ✓ Reset or rebuild in seconds
- ✓ Spin up many, side by side

*A virtual network behaves like the real thing — students can ping, route, and capture traffic — but it lives entirely inside their computer.*

# What is Docker?

Docker bundles an app with everything it needs to run — code, libraries, and settings — into a **container** that runs the same way on any machine.

**Think shipping containers:** standardized boxes that hold their contents intact and run identically on a laptop, a server, or the cloud — no "works on my machine" surprises.



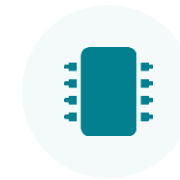
## Self-contained

Ships with all dependencies inside — no manual setup or version conflicts on the host.



## Starts instantly

Spins up in seconds — perfect for scaling, testing, and rapid deployment.



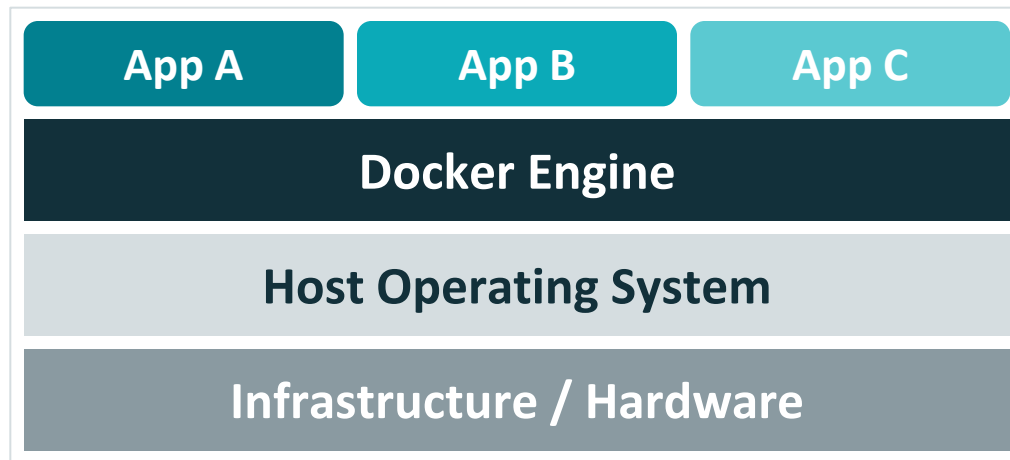
## Very lightweight

Shares the host OS kernel, so dozens can run on one machine with minimal overhead.

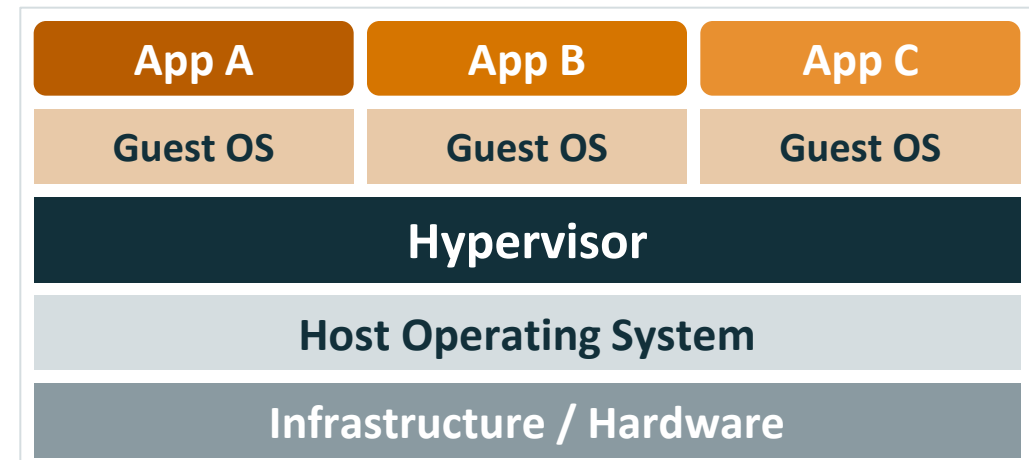
# Docker vs. Virtual Machines

Both isolate apps from the host, but they do it at very different levels of the stack.

## DOCKER (CONTAINERS)



## VIRTUAL MACHINES



### SIZE

**Docker:** MBs

**VM:** GBs

### STARTUP

**Docker:** Seconds

**VM:** Minutes

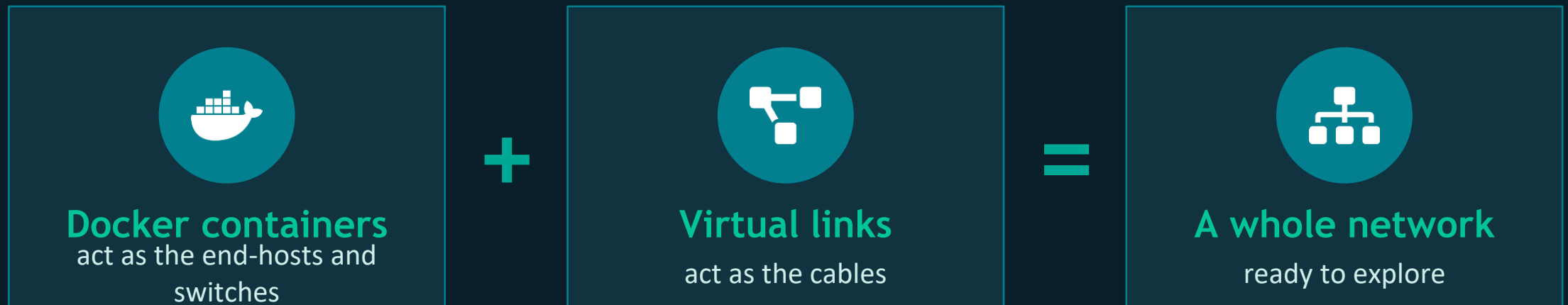
### ISOLATION

**Docker:** Process-level

**VM:** Full OS

# So what is Containerlab?

It combines the two ideas you just saw.



**Containerlab is a free tool that builds virtual networks out of containers.** You describe the network you want in one short text file, and it wires everything up for you in seconds.

# Why is it useful for a course?



## No special hardware

Every student gets their own network on a laptop or a shared server. No lab full of equipment needed.



## Fast to set up and reset

A network appears in seconds and can be wiped clean just as fast — perfect for trying things and starting over.



## Safe to experiment

Mistakes cost nothing. Students can break things, learn, and rebuild without affecting anyone else.



## Real, transferable skills

Students use the same tools and commands that real network engineers use every day.

# Hold on to this one idea

**A network is just boxes and links.**

Containerlab lets you create them in software, from a single file, for free.

## What comes next in the tutorial

- Installing Containerlab on your computer
- Building and running your first topology
- Three ready-to-use assignment ideas

*Now that the picture is clear, let's get hands-on.*



# YAML Topology Files

Describing your network in a single text file

# What is YAML?

## YAML = Yet Another Markup Language

A simple, human-readable format for describing structured data — no braces, no quotes (usually).

### ► Indentation = Structure

Two spaces per level. Never use tabs. Child items are indented under their parent.

### ► key: value

Every field is key: value. No equals signs, no semicolons. A colon followed by a space.

### ► - list items

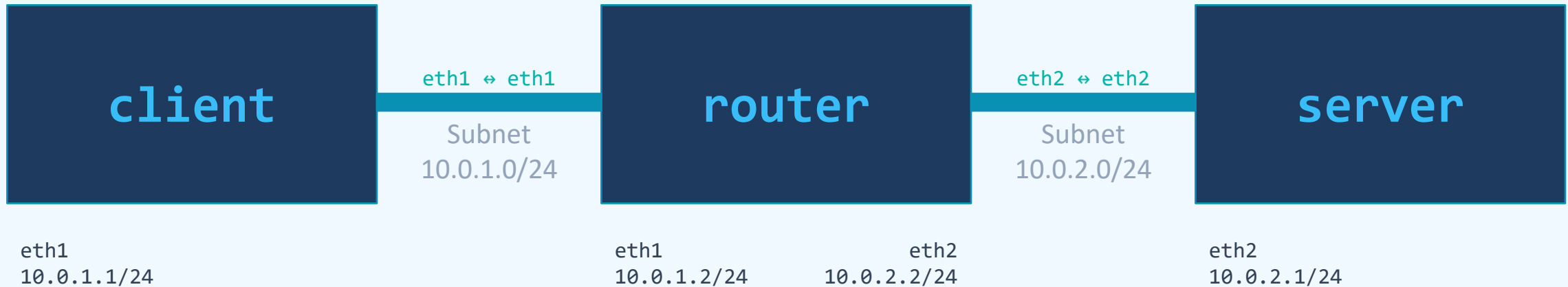
A dash followed by a space starts a list item. Used for nodes, links, exec commands.

```
name: my-lab      # top-level name

topology:         # everything lives here
  nodes:          # define devices
    router:       # a node called "router"
      kind: linux  # its type
  links:           # define connections
    - endpoints:  # a link entry
      - "router:eth1"
```

# What the topology looks like

Three nodes in a line — client ↔ router ↔ server



## How routing works in this topology

client → knows route to 10.0.2.0/24 via 10.0.1.2 (the router's left interface)

router → has `ip_forward=1` turned on, so it forwards packets between subnets

server → knows route to 10.0.1.0/24 via 10.0.2.2 (the router's right interface)

# The linear.yaml – Full File Walkthrough

```
name: linear

topology:
  nodes:
    client:
      kind: linux
      image: alpine:latest
      exec:
        - ip addr add 10.0.1.1/24 dev eth1
        - ip route add 10.0.2.0/24 via 10.0.1.2

    router:
      kind: linux
      image: alpine:latest
      exec:
        - sysctl -w net.ipv4.ip_forward=1
        - ip addr add 10.0.1.2/24 dev eth1
        - ip addr add 10.0.2.2/24 dev eth2

    server:
      kind: linux
      image: alpine:latest
      exec:
        - ip addr add 10.0.2.1/24 dev eth2
        - ip route add 10.0.1.0/24 via 10.0.2.2

  links:
    - endpoints: ["client:eth1", "router:eth1"]
    - endpoints: ["router:eth2", "server:eth2"]
```

## name:

A label for this lab. Containerlab prefixes containers: clab-linear-client, clab-linear-router ...

## kind: linux

Use Alpine Linux as a lightweight host. Could also be nokia\_srlinux, cisco\_xrd, etc.

## image:

The Docker image to pull. alpine:latest is tiny (~5 MB) and has basic networking tools.

## exec:

Commands run inside the container right after boot — sets IP addresses and routes.

## links:

Wires two nodes together. Each endpoint is node:interface — like plugging in a cable.

# Running the Lab

Deploy, inspect, enter, and destroy your topology

# Deploying and Inspecting the Lab

## 1 Deploy the topology



```
sudo clab deploy -t linear.yaml
```

Containerlab reads the YAML, pulls Docker images, creates containers, and wires the virtual links. Takes ~10–30 seconds on first run (image download).

## 2 Inspect the running lab



```
sudo clab inspect -t linear.yaml
```

| # | Name               | Kind  | State   | IPv4 | Image         |
|---|--------------------|-------|---------|------|---------------|
| 1 | clab-linear-client | linux | running | N/A  | alpine:latest |
| 2 | clab-linear-router | linux | running | N/A  | alpine:latest |
| 3 | clab-linear-server | linux | running | N/A  | alpine:latest |

### clab deploy

Creates and starts all containers. Uses the YAML topology file as the blueprint.

### clab inspect

Shows a table of all running nodes: names, states, management IPs.

### Container names

Always prefixed with `clab-<labname>-<nodename>`. You use these exact names in docker commands.

# Entering Containers – SSH and exec

There are two ways to get a shell inside a running Containerlab node:

## Method 1: docker exec (recommended)



```
sudo docker exec -it clab-linear-client sh
```

**docker exec** runs a command inside an already-running container.

**-it** = interactive terminal (keeps stdin open, allocates a TTY).

**sh** = the shell to run. Use bash if available, sh otherwise.

## Method 2: SSH (for nodes with sshd)



```
ssh admin@clab-linear-router
```

Some node kinds (Nokia SR Linux, Arista cEOS, etc.) run an SSH daemon and have a management IP. Use ssh for them — same as connecting to real hardware. alpine:latest nodes don't run sshd by default, so use docker exec.

Once you're inside the container ...



```
# You're now inside clab-linear-client
/ # ip addr show eth1           # see IP address on eth1
/ # ip route show               # see the routing table
/ # ping 10.0.2.1               # ping the server
/ # traceroute 10.0.2.1         # trace the path
/ # exit                       # leave the container
```

# Testing Connectivity – Ping & Traffic Capture

## Ping from client to server (non-interactive)



```
sudo docker exec clab-linear-client ping -c 4 10.0.2.1
```

**docker exec** (without -it) runs a command and returns output — no interactive shell needed.  
**-c 4** = send exactly 4 ICMP echo packets, then stop.

```
PING 10.0.2.1 (10.0.2.1): 56 data bytes
64 bytes from 10.0.2.1: seq=0 ttl=63 time=0.312 ms
64 bytes from 10.0.2.1: seq=1 ttl=63 time=0.287 ms
--- 10.0.2.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss
```

## Capture traffic on the router (in a second terminal)



```
sudo docker exec clab-linear-router tcpdump -i eth1 -n
```

**-i eth1** watch interface eth1 • **-n** no DNS lookup (show raw IPs) • Press **Ctrl+C** to stop



### Why ttl=63?

The client starts with TTL 64. The router decrements it by 1 when it forwards. Server receives TTL 63 — proving the packet went through the router!



# Tearing Down – clab destroy

When you're done, destroy the lab in one command:

```
sudo clab destroy -t linear.yaml
```

## Stops all containers

Every Docker container in the topology is stopped and removed.

## Removes virtual links

All software cables (veth pairs) connecting the nodes are deleted.

## Cleans up

The clab-<name> directory with management configs is removed from disk.

 Any data created inside containers is lost on destroy. If you need to save configs, copy them to the host before running clab destroy.

To start fresh:

```
sudo clab deploy -t linear.yaml
```

# Containerlab Quick-Reference Cheatsheet

```
sudo clab deploy -t FILE.yaml
```

Start the lab described in FILE.yaml

```
sudo clab destroy -t FILE.yaml
```

Stop and remove all containers + links

```
sudo clab inspect -t FILE.yaml
```

Show table of running nodes and their IPs

```
sudo docker exec -it clab-LAB-NODE sh
```

Open interactive shell inside NODE

```
sudo docker exec clab-LAB-NODE ping -c 4 IP
```

Ping from NODE to IP (non-interactive)

```
sudo docker exec clab-LAB-NODE tcpdump -i IFACE -n
```

Capture packets on IFACE inside NODE

```
ssh admin@clab-LAB-NODE
```

SSH in (nodes with sshd, e.g. Nokia/Arista)

```
ip addr add IP/PREFIX dev IFACE
```

Assign IP to interface (run inside container)

```
ip route add NET via GW
```

Add a static route (run inside container)

```
sysctl -w net.ipv4.ip_forward=1
```

Enable IP forwarding — turn node into a router

# IP Rules & Routing Tables – Reading the Network State

These three commands tell you exactly what the kernel will do with each packet — always run them first when debugging.

## ① `ip addr show`

Shows every interface and its assigned IP address.



```
ip addr show eth1
```

```
4: eth1@if5:  
    <BROADCAST,MULTICAST,UP>  
    link/ether aa:c1:ab:12:34:56  
    inet 10.0.1.1/24 scope global  
        eth1
```

### Key fields:

- inet → the IPv4 address and prefix
- /24 → subnet mask (255.255.255.0)
- UP → interface is active

## ② `ip route show`

Shows the kernel routing table — how packets are forwarded.



```
ip route show
```

```
10.0.1.0/24 dev eth1 proto kernel  
    scope link src 10.0.1.1  
10.0.2.0/24 via 10.0.1.2 dev eth1
```

### Key fields:

- dev eth1 → exit via this interface
- via 10.0.1.2 → next-hop (gateway)
- proto kernel → auto-added when IP assigned

## ③ `ip rule show`

Shows routing policy rules — which table to consult per packet.



```
ip rule show
```

```
0:    from all lookup local  
32766: from all lookup main  
32767: from all lookup default
```

### Key fields:

- priority 0 → local (loopback, own IPs)
- priority 32766 → main table (your routes)
  - lookup main → use 'ip route show' table

# Live Traffic Analysis – Ping on One Node, tcpdump on Another

Open two terminals. Start tcpdump first, then ping. You'll watch packets arrive in real time.

## 1 Terminal 1 — Start capture on router

### Terminal 1 — router

```
sudo docker exec clab-linear-router tcpdump -i eth1 -n -v
```

**-i eth1** Listen on interface eth1 (client-facing side of the router)

**-n** Don't resolve IPs to hostnames — shows raw addresses

**-v** Verbose: show TTL, checksum, packet length in each line

What you see in each terminal:

```
tcpdump: listening on eth1 ...
IP 10.0.1.1 > 10.0.2.1: ICMP echo req, ttl 64
IP 10.0.2.1 > 10.0.1.1: ICMP echo rep, ttl 63
IP 10.0.1.1 > 10.0.2.1: ICMP echo req, ttl 64
IP 10.0.2.1 > 10.0.1.1: ICMP echo rep, ttl 63
```

← router sees both directions of traffic

## 2 Terminal 2 — Ping from client to server

### Terminal 2 — client

```
sudo docker exec clab-linear-client ping -c 5 10.0.2.1
```

**-c 5** Send exactly 5 packets then stop

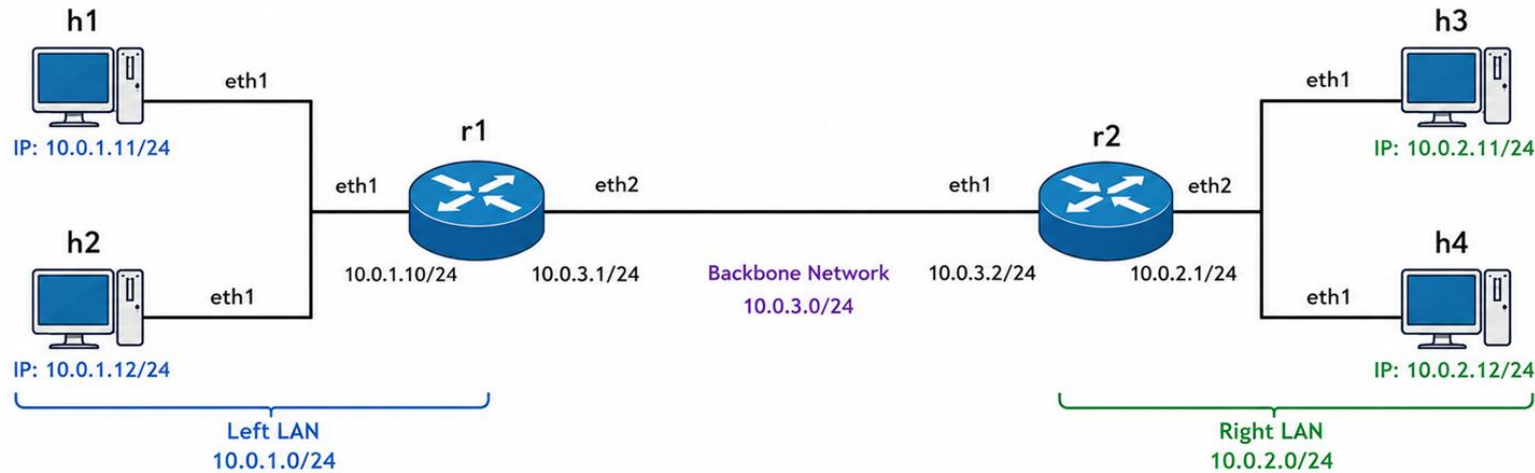
**10.0.2.1** The server's IP — traffic must route through the router

```
PING 10.0.2.1: 56 data bytes
64 bytes from 10.0.2.1: ttl=63 time=0.3 ms
64 bytes from 10.0.2.1: ttl=63 time=0.2 ms
64 bytes from 10.0.2.1: ttl=63 time=0.3 ms
5 packets tx, 5 received, 0% packet loss
```

← ttl=63 proves packet passed through the router

## 3 Terminal 1 — Watch packets arrive

# Assignment – Build the Dumbbell Topology



## Legend

- Host (h1–h4)
- Router (r1, r2)
- Backbone link
- Access link

## Your Tasks

- 1 Write the YAML** Define all 6 nodes (h1–h4, r1, r2) with correct kinds, images, and exec commands to set IPs and routes. Define 5 links matching the diagram above.
- 2 Deploy & verify** Run clab deploy. Use ip addr show and ip route show inside each node to confirm all addresses and routes are correct.
- 3 Connectivity tests** Ping h3 from h1. It must pass through both r1 and r2. Check TTL — it should be 62 (decremented twice). Trace the path with traceroute.
- 4 Live capture** While pingging h3 from h1, run tcpdump on the backbone link (r1 eth2). Confirm you see traffic on the 10.0.3.0/24 subnet.